

Module 04 Assignment Guide

Exploring Spiking Neurons with Wolfram Language

Houston City College • Spring 2026

Before You Begin

This guide walks you through building an interactive spiking neuron simulation using Wolfram Language.

You will NOT find complete solutions here — you will find explanations, starter code, hints, and tasks where you write the code yourself.

Answer all Knowledge Checks and Reflections in text cells in your notebook.

Platform: Wolfram Cloud (wolframcloud.com) or Mathematica Desktop

File name: A04_FirstName_LastName_ITAI4374.nb or .pdf

Wolfram Quick Reference

Shift+Enter Run a cell

(* comment *) Add comments to your code

?FunctionName Look up any function's documentation

NDSolve[...] Solve differential equations numerically

Plot[...] Create a 2D plot

Manipulate[...] Create interactive controls with sliders

Functions always start with a capital letter. Arguments go in square brackets [].

Part A: Solving the LIF Equation with NDSolve

Step 1: Verify Your Setup

Open Wolfram Cloud or Mathematica and run this in a new cell to confirm everything works:

```
Print["Wolfram Language ready!"]
Print[$Version]
```

You should see a version number. If you are in Wolfram Cloud, the interface may look slightly different from Desktop Mathematica, but the code is identical.

Step 2: Understand the LIF Equation in Wolfram

Wolfram Language excels at solving differential equations directly. Instead of writing a loop like in Python, we can give Wolfram the LIF equation and let NDSolve handle it.

The LIF equation as a differential equation:

```
(* The LIF differential equation: *)
(* tau * V'[t] == -(V[t] - Vrest) + R * I[t] *)
(*
(* tau = membrane time constant (seconds) *)
(* Vrest = resting potential (mV) *)
(* R = membrane resistance (MOhm) *)
(* I[t] = input current (nA) *)
(*
(* When V[t] >= Vthresh, neuron spikes and resets *)
```

The challenge: NDSolve can't handle the discontinuous reset directly. We will use WhenEvent to detect threshold crossing and reset the voltage.



TIP

WhenEvent is a powerful Wolfram feature that lets you say: “when this condition becomes true, do this action.” Perfect for spike detection and reset.

Step 3: Define Neuron Parameters

Set up your parameters. Create a new cell:

```
(* ===== Neuron Parameters ===== *)
tau    = 0.020;    (* Membrane time constant: 20 ms *)
Vrest  = -70.0;    (* Resting potential in mV *)
Vthresh = -55.0;   (* Spike threshold in mV *)
Vreset = -75.0;    (* Reset potential after spike *)
R      = 1.0;      (* Membrane resistance, normalized *)

(* ===== Input Current ===== *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 <= t <= 0.4, 18.0, 0.0];

Print["Parameters defined."]
```

KNOWLEDGE CHECK

Answer these in a text cell below your code.

1. The input current is defined as a function of time using If[]. In plain English, what does Iinput[t] return at t = 0.03? At t = 0.2? At t = 0.5?
2. Why do we use := (delayed assignment) for Iinput instead of = (immediate assignment)?

Step 4: Solve the LIF Equation

YOUR TASK 1: Use NDSolve with WhenEvent

Write the NDSolve call to solve the LIF equation. Use WhenEvent to detect when V[t] crosses the threshold and reset it.

```
(* Solve the LIF equation with spike reset *)
spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
```

```

(* TODO 1: Write the LIF differential equation *)
(* tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t] *)
(* YOUR CODE HERE *)

(* Initial condition: start at resting potential *)
V[0] == Vrest,

(* TODO 2: WhenEvent for spike detection and reset *)
(* When V[t] reaches Vthresh: *)
(*   - Reset V[t] to Vreset *)
(*   - Increment spikeCount *)
(*   - Append t to spikeTimes *)
(* Hint: WhenEvent[V[t] >= Vthresh, *)
(*   {V[t] -> Vreset, *)
(*     spikeCount++, *)
(*     AppendTo[spikeTimes, t]}] *)
(* YOUR CODE HERE *)
},
V,
{t, 0, 0.5},
DiscreteVariables -> {spikeCount}
];

Print["Neuron fired ", spikeCount, " spikes"]
Print["Spike times: ", spikeTimes]

```

TIP

If you get an error about DiscreteVariables, make sure spikeCount is initialized before the NDSolve call.

WhenEvent needs the condition and actions wrapped in curly braces { }.

The -> in V[t] -> Vreset means “replace V[t] with Vreset.”

CHECKPOINT

Your neuron should fire roughly 10–20 spikes. If you get 0 spikes, check that your differential equation has the correct sign. If you get errors, double-check your WhenEvent syntax.

Step 5: Plot the Membrane Potential

YOUR TASK 2: Visualize the Solution

Use Plot to show the membrane potential over time. Add a dashed line at the threshold.

```
(* TODO 3: Plot the membrane potential *)
(* Hint: Plot[V[t] /. sol, {t, 0, 0.5}] extracts the solution *)
(* Use Epilog to add a dashed threshold line: *)
(*   Epilog -> {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]} *)
(* Add labels: AxesLabel, PlotLabel *)
(* YOUR CODE HERE *)
```

CHECKPOINT

You should see a sawtooth-like pattern: voltage rises, hits threshold, resets, rises again. The dashed red line shows the threshold.

KNOWLEDGE CHECK

Answer these in a text cell below your code.

1. Look at the shape of V between spikes. Is it a straight line or a curve? Why?
2. What happens to V after the current turns off ($t > 0.4$)? Describe what you see and explain it in terms of the LIF equation.

Part B: Interactive Manipulate[] Dashboard

This is where Wolfram really shines. Manipulate[] lets you create sliders that update your simulation in real time. You can explore what happens when you change neuron parameters without rewriting any code.

Step 6: Build the Dashboard

YOUR TASK 3: Create an Interactive LIF Explorer

Wrap your NDSolve + Plot code inside a Manipulate[] with sliders for at least 3 parameters: threshold, time constant, and input current.

```
Manipulate[
  Module[{sol, sc = 0, st = {}},

    (* TODO 4: Put your NDSolve code here, but replace *)
    (* the fixed parameter values with the slider variables: *)
    (* Vthresh -> thresh *)
    (* tau -> tauVal *)
    (* The constant 18.0 in Iinput -> currentVal *)
    (*
    (* Define a local input current function: *)
    (* iFunc[t_] := If[0.05 <= t <= 0.4, currentVal, 0.0]; *)
    (*
    (* Then NDSolve with these local variables... *)
    (* YOUR CODE HERE *)

    (* TODO 5: Plot the result *)
    (* Include in the PlotLabel: how many spikes fired *)
    (* Example: PlotLabel -> StringForm["Spikes: `", sc] *)
    (* YOUR CODE HERE *)

  ],

  (* Sliders *)
  {{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
  {{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
  {{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]
```

TIP

`Module[{local variables}, code]` creates a local scope so your Manipulate doesn't interfere with variables defined outside it.

If Manipulate is slow, try reducing the simulation time to 0.3 seconds or making the slider step sizes larger.

You can add more sliders later (`Vreset`, `R`, current duration) if you want to explore further.

CHECKPOINT

Move the sliders and watch the plot update. Lowering the threshold should increase firing rate.

Increasing current should increase firing rate. Increasing tau should slow down the response.

KNOWLEDGE CHECK

Answer these in a text cell below your code.

1. Using your dashboard, find the minimum input current needed to make the neuron fire at least one spike (with default threshold and tau). This is called the rheobase. What value did you find?
2. Set the current to 25 nA. Now slowly increase the threshold from -65 to -45 mV. At what threshold does the neuron stop firing? Why?
3. Set threshold to -55 and current to 20. Compare $\tau = 0.010$ vs $\tau = 0.050$. Which produces more spikes? Which produces a smoother membrane potential curve?

REFLECTION

The Manipulate[] dashboard lets you explore neuron behavior interactively in a way that static code cannot. Describe one insight you gained from adjusting the sliders that you did NOT get from reading the booklet. How did the visual feedback change your understanding?

Part C: Firing Rate Curve (f–I Curve)

The f–I curve (frequency vs. input current) is one of the most important characterizations of a neuron. It shows how spike frequency increases with input strength. This is how neuroscientists measure a neuron's "transfer function."

Step 7: Generate the f–I Curve

YOUR TASK 4: Sweep Current and Count Spikes

Write a loop that runs NDSolve for input currents from 0 to 35 nA. For each current, count how many spikes the neuron fires. Plot the result.

```
(* TODO 6: Generate f-I curve data *)
(* Create a list of {current, spikeCount} pairs *)
(*
(* Hint: Use Table to loop over current values: *)
(*   fiData = Table[ *)
(*     Module[{sc = 0, ...}, *)
(*       sol = NDSolve[...with Icurr as the current...]; *)
(*       {Icurr, sc} *)
(*     ], *)
(*     {Icurr, 0, 35, 1} *)
(*   ]; *)
(*
(* Then plot: *)
(*   ListLinePlot[fiData, *)
(*     AxesLabel -> {"Input Current (nA)", "Spike Count"}, *)
(*     PlotLabel -> "f-I Curve", *)
(*     PlotMarkers -> Automatic] *)
(*
(* YOUR CODE HERE *)
```

TIP

This reuses your NDSolve code from Part A. The only change is that the input current value comes from the Table loop instead of being fixed.

Use a simulation time of 0.5 seconds so you have enough time for spikes to accumulate.

If it runs slowly, reduce the current range or increase the step size.

CHECKPOINT

You should see a curve that starts at 0 (no spikes for low current), then rises. The curve should be roughly linear in the middle range but may flatten at high currents.

KNOWLEDGE CHECK

Answer these in a text cell below your code.

1. At what input current does your neuron first start firing? This is related to the rheobase you found with the dashboard.
2. Is the $f-I$ curve a straight line, or does it curve? What would a perfectly straight line mean about how the neuron encodes stimulus intensity?
3. Real neurons often show a “saturation” effect at high currents (firing rate plateaus). Does your LIF model show this? Why or why not?

Part D: Encoding a Signal as Spikes

Now let's use your LIF neuron as an encoder. Instead of injecting constant current, we will feed it a time-varying signal (a sine wave) and observe how it converts that signal into a spike train.

Step 8: Rate Coding with a Varying Signal

YOUR TASK 5: Encode a Sine Wave as Spikes

Modify your LIF simulation to use a sine wave as input current. Plot both the input signal and the resulting spike train.

```
(* Define a sine wave input current *)
(* The sine wave oscillates, so we shift it to be always positive *)
Isine[t_] := 15.0 + 12.0 * Sin[2 Pi * 5 * t];
(* This gives a 5 Hz sine wave centered at 15 nA, ranging from 3 to 27 nA *)

(* TODO 7: Run NDSolve with Isine[t] as the input current *)
(* Use the same LIF equation and WhenEvent as Part A *)
(* Simulation time: 0 to 1.0 seconds *)
(* YOUR CODE HERE *)

(* TODO 8: Create a two-panel plot *)
(* Top: membrane potential V[t] *)
(* Bottom: the input current Isine[t] *)
(* Hint: Use GraphicsColumn[{plot1, plot2}] or Grid *)
(* YOUR CODE HERE *)
```

CHECKPOINT

You should see bursts of spikes when the sine wave is high (peak) and fewer or no spikes when the sine wave is low (trough). The spike density follows the input signal shape.

REFLECTION

Look at your spike train alongside the sine wave input. The neuron is acting as an analog-to-digital converter: it takes a continuous signal and converts it into discrete spikes. How well does the spike pattern represent the original signal? What information is preserved? What is lost?

Part E: Experiments

Run at least 3 of the experiments below. For each, use your Manipulate[] dashboard or modify your code, observe what happens, and document your findings in a text cell.

WARNING

For each experiment: create a new cell, make the change, run it, and take a screenshot or keep the output. Write your observations in a text cell below. Do NOT overwrite your working code from earlier parts.

Experiment Menu (choose at least 3)

Experiment 1: Threshold Sweep

Using your Manipulate[] dashboard, systematically change the threshold from -65 to -45 in steps of 5. For each setting, record the spike count. Create a table or plot of threshold vs. spike count.

Experiment 2: Time Constant Comparison

Generate 3 membrane potential plots side by side: $\tau = 0.010$, $\tau = 0.020$, $\tau = 0.050$. Use GraphicsRow[{plot1, plot2, plot3}] to display them together. Describe how the “shape” of V changes.

Experiment 3: Noisy Input

Replace your constant current with a noisy version: $I_{\text{noisy}}[t] := 18.0 + \text{RandomReal}[-8, 8]$. Run the simulation several times. Is the spike pattern the same each time? Why or why not?

Experiment 4: Refractory Period

After a spike, real neurons have a brief period where they cannot fire again (refractory period). Modify your WhenEvent to include a minimum inter-spike interval of 3 ms. How does this change the maximum firing rate?

 TIP

Hint: Add a variable `lastSpike` and check that `t - lastSpike > 0.003` in your `WhenEvent` condition.

Experiment 5: Two Different f – I Curves

Generate f – I curves for two different time constants (e.g., $\tau = 0.010$ and $\tau = 0.040$) and plot them on the same graph. How does τ affect the neuron's sensitivity to input current?

Experiment 6: Frequency Encoding

Change the sine wave frequency from 5 Hz to 2 Hz, then 10 Hz, then 20 Hz. At what frequency can the neuron no longer follow the oscillations? What does this tell you about the neuron's temporal resolution?

 For Each Experiment, Write:

1. What parameter did you change, and to what values?
2. What did you expect to happen?
3. What actually happened? (Include the output or screenshot.)
4. Why does this make sense given the LIF equation?

Part F: Final Reflection

Create a text cell with the heading “Final Reflection” and answer the following questions thoughtfully. These connect your hands-on Wolfram experience to the neuroscience concepts from the booklet.

REFLECTION

1. Using your Manipulate[] dashboard, describe what happens as you gradually increase input current from below threshold to well above it. At what point does the neuron start firing? What does this threshold behavior tell you about how biological neurons make “decisions”?

REFLECTION

2. Examine your f–I curve. Is the relationship linear? What biological significance does the shape of this curve have for how the brain encodes stimulus intensity?

REFLECTION

3. How does changing the membrane time constant affect firing? In biological terms, what types of neurons might have short vs. long time constants, and what would each be good at?

REFLECTION

4. Compare the LIF model to what you learned about real neurons in the booklet. What does the model capture well? What important features does it miss? (Think about: dendrite morphology, multiple ion channels, neurotransmitter dynamics, adaptation.)

REFLECTION

5. Wolfram’s Manipulate[] gave you real-time interactive exploration. How did this change your understanding compared to just reading about neurons? What was the most surprising thing you discovered by moving the sliders?

You're Done!

Make sure your notebook contains: all code cells with outputs visible, all Knowledge Check answers, all Experiment observations, and the Final Reflection.

From Wolfram Cloud: File → Download as → PDF

From Mathematica Desktop: Save as .nb or File → Print to PDF

File name: A04_FirstName_LastName_ITAI4374.nb or .pdf

Submit your single file to Canvas.